Assignment 4: Maximum Concurrent Flow and Network Design

CS 422: Computer Networks, Fall 2026

Due: November 6, 2026

1 Uniform traffic matrix

Here $T_{ij} = d/(n-1)$ for every $i \neq j$.

1.1 Idea

Two things limit λ, the largest fraction of demand that the network can carry at the same time. First, every hop uses up link capacity, and the network has only nd units, so shorter routes are better for maximizing flow. Second, λ is defined by the busiest link, thus its imperative that all of the links are equally busy.

The algorithm (1) computes a ceiling that no topology can exceed, (2) searches a list of symmetric topologies that are both short and balanced, (3) stops if the ceiling is reached, and (4) otherwise solves the problem over all possible topologies.

1.2 Step 1: a ceiling that holds for every topology

We are trying to see at its most compact, how tight we can make the hops of the other $n-1$ nodes: d nodes at distance 1, d^2 at distance 2, etc. If k is the number of levels needed (where smallest k with $d + d^2 + \cdots + d^k \geq n-1$), then $r = (n-1) - (d + \cdots + d^{k-1})$ is the number of nodes on the last, partially filled level, and

$$M \;=\; \sum_{\ell=1}^{k-1} \ell\, d^{\ell} \;+\; k\, r$$

is the total number of hops from a node to all other nodes in the best case.

Lemma 1 (Ceiling)**.** *For every topology with d links out and d links in per node, $\lambda^\star \leq \mathrm{UB} := (n-1)/M$.*

Proof. We know that the sum of distances from any node to the others is at least M. Traffic from i to j must cross $\mathrm{dist}(i,j)$ links, and each hop uses one unit of capacity, so the total capacity used is at least $\geq \lambda \frac{d}{n-1}\, n\, M$. The network has only nd units of capacity, so $\lambda \frac{d\, n\, M}{n-1} \leq nd$, i.e. $\lambda \leq (n-1)/M$. □

1.3 Algorithm

A *circulant* topology $C_n(S)$ for an offset list $S \subseteq \{1, \ldots, n-1\}$, $|S| = d$, has one link from node i to node $(i+s) \bmod n$ for every i and every $s \in S$. Every node then looks the same (the same distances, the same d link types), so only d offsets have to be chosen and the load can be balanced across just d link types.

Algorithm 1 BestUniformTopology(n, d)

Step 1: ceiling
k, r, M as defined above; UB $\leftarrow (n-1)/M$
Step 2: search the symmetric topologies
$T_{ij} \leftarrow d/(n-1)$ for all $i \neq j$
$\mathcal{C} \leftarrow$ all offset lists S of d distinct values from $\{1, \ldots, n-1\}$ for which $C_n(S)$ is strongly connected
for each $S \in \mathcal{C}$: $\sigma(S) \leftarrow$ sum of distances from node 0 ▷ one BFS
sort $\mathcal{C}$ by $\sigma(S)$, smallest first; best $\leftarrow 0$
for all $S \in \mathcal{C}$ in that order **do**
  if $(n-1)/\sigma(S) \leq$ best **then break** ▷ nothing later can win
  end if
  $\lambda \leftarrow$ ConcurrentFlow$(C_n(S), T)$ ▷ a linear program
  if $\lambda >$ best **then** best $\leftarrow \lambda$, $S^\star \leftarrow S$
  end if
end for
Step 3: certified by the ceiling?
if best = UB **then return** $C_n(S^\star)$ ▷ provably optimal
end if
Step 4: exact solve over all topologies
choose link counts x_{ij} (whole numbers) and a routing to **maximize** λ subject to:
  every node has exactly d links out and d links in;
  each node v receives λT_{sv} from every source s;
  on every (i, j), total traffic $\leq x_{ij}$.
return x

1.4 Why the result is guaranteed to be the best

The algorithm returns for only one of two reasons, and each is airtight by itself.

It returns at Step 3. By Lemma 1 if the topology we found matches UB, then no topology can do better, so we can just return that.

It returns at Step 4. Since this iterates through all possible legal toplogies and finds the best routing, its answer does return the best definition.

2 Arbitrary traffic matrix in the hose set ($n = 8$, $d = 4$)

The algorithm has the same shape as in Part 1: a ceiling, a quick construction, a check against the ceiling, and an exact solve if the check fails.

2.1 Idea

If T was just a matrix of whole numbers, then the tooplogy could just carry that demand on direct links since each node is always just guranteed 4 units of input and output capacity. It is fractional demands that we have to account for, and how we can round these demands into whole numbers.

2.2 Step 1: a ceiling that holds for every topology

We link a pair (i, j) *linked* if $x_{ij} \geq 1$. Le's set C as the maximum total demand $\sum T_{ij}$ over linked pairs that any choice of linked pairs can achieve, where each node may be linked to at most d destinations and from at most d sources. This is a maximum-weight assignment (degree-constrained matching) problem and is solved exactly in polynomial time.

Lemma 2 (Ceiling for a given T)**.** *For every topology,* $\lambda^\star \leq \mathrm{UB}(T) := \dfrac{nd}{2\sum_{ij} T_{ij} - C}$.

Proof. Traffic on a linked pair needs at least 1 hop, and traffic on a pair with no direct link needs at least 2. Each hop uses one unit of capacity, so the capacity used is at least $\lambda\big(\sum T + \sum_{\text{unlinked}} T\big) = \lambda\big(2\sum T - \text{covered}\big)$, where "covered" is the demand on linked pairs. Whatever the topology (parallel links only reduce the number of distinct neighbours), a node has at most d distinct out-neighbours and in-neighbours, so covered $\leq C$. The network has nd units of capacity, so $\lambda(2\sum T - C) \leq nd$. □

Remark 1. For $n = 8$, $d = 4$ and a topology with at most one link per pair, every pair is at most 2 hops apart: if i has no direct link to j, then the 4 out-neighbours of i and the 4 in-neighbours of j all lie among the other 6 nodes, so at least 2 nodes are in both sets and give two 2-hop routes. Hence an unlinked pair costs exactly one extra hop, and the hop count in the proof above is attained, so minimizing it (maximizing C) is the right target for the construction.

2.3 Algorithm

Algorithm 2 BestTopologyForHose(T), $n = 8$, $d = 4$

Step 1: ceiling
$C \leftarrow$ largest demand coverable by direct links (at most d out and d in per node, one link per pair)
UB $\leftarrow nd/(2\sum T - C)$
Step 2: build a topology from the demands
$x_{ij} \leftarrow \lfloor T_{ij} \rfloor$ for all $i \neq j$ ▷ whole part of each demand
give the remaining links to the pairs with the most unmet demand ▷ an assignment problem
repeat
  swap two links $(a \to b,\ c \to e) \mapsto (a \to e,\ c \to b)$; keep the swap if λ improves ▷ degrees unchanged
until no swap improves λ
best $\leftarrow \lambda(x)$ ▷ ConcurrentFlow(x, T), a linear program
Step 3: certified by the ceiling?
if best = UB **then return** x ▷ provably optimal
end if
Step 4: exact solve over all topologies
choose link counts x_{ij} (whole numbers) and a routing to **maximize** λ subject to:
  every node has exactly d links out and d links in;
  each node v receives λT_{sv} from every source s;
  on every (i, j), total traffic $\leq x_{ij}$.
return x

2.4 How the topology adapts to T, and why the result is the best

Adaptation. Links are placed where the demand is: pairs with large demand get direct links (several if the demand exceeds 1), and the swaps in Step 2 then move links to relieve the busiest links and shorten the worst routes. A different T gives a different topology.

It returns at Step 3. Lemma 2 holds for *every* topology, so a topology that achieves $\mathrm{UB}(T)$ cannot be beaten.

It returns at Step 4. The program ranges over every legal topology and its best routing at once, so its answer is the best by definition. Its three conditions describe the problem exactly: the degree condition says the topology is legal, the delivery condition says each source's traffic reaches each destination in the scaled amount, and the capacity condition says traffic uses only links that exist, one unit per link. This holds for every T in $\mathcal{T}_{\text{hose}}$, since T enters only as data.

Where the guarantee comes from. The ceiling is exact for some matricesw but it can be loose for matrices with uneven demands, because it counts a pair with a direct link as fully covered even though one link carries only one unit. For such matrices Step 3 fails and the answer comes from Step 4.